Лабораторная работа 5 «Оптимизация непрерывных функций»

Эта лабораторная работа посвящена решению непрерывных задач оптимизации с помощью генетических алгоритмов (оптимизируем функцию Eggholder и функцию Химмельблау).

В этой лабораторной работе нам понадобится Python 3 и следующие библиотеки:

- ✓ deap;
- ✓ numpy;
- ✓ matplotlib;
- ✓ seaborn.

ХРОМОСОМЫ И ГЕНЕТИЧЕСКИЕ ОПЕРАТОРЫ ДЛЯ ЗАДАЧ С ВЕЩЕСТВЕННЫМИ ЧИСЛАМИ

Будем рассматривать задачи с непрерывным пространством состояний, когда решения представлены вещественными числами с плавающей точкой.

Рассмотрим две хромосомы: $[x_1, x_2, x_3]$ и $[y_1, y_2, y_3]$. Применение операции скрещивания к соответственным генам следующим образом порождает потомка:

- o_1 – результат применения оператора скрещивания к x_1 и y_1;
- o_2 – результат применения оператора скрещивания к x_2 и y_2;
- o_3 – результат применения оператора скрещивания к x_3 и y_3.

Аналогично оператор мутации применяется к каждому элементу по отдельности, поэтому o_1, o_2 и o_3 могут быть подвергнуты мутации.

Перечислим некоторые распространенные вещественные операторы.

- **Скрещивание смешением** (Blend Crossover – **BLX**), когда каждый потомок случайным образом выбирается из интервала, образованного родителями:

  $$[parent_1 - \alpha(parent_2 - parent_1), parent_2 + \alpha(parent_2 - parent_1)].$$

  Коэффициент α обычно принимается равным 0.5, поэтому интервал выбора в два раза шире интервала между родителями.
- **Имитация двоичного скрещивания** (Simulated Binary Crossover – SBX), когда два потомка порождаются по следующей формуле, гаран-

тирующей, что среднее значение потомков равно среднему значению родителей:

$$offsping_1 = ½ [(1 + β]parent_1 + (1 - β)parent_2]);$$
$$offsping_2 = ½ [(1 - β]parent_1 + (1 + β)parent_2]).$$

Коэффициент β, называемый **коэффициентом распределения**, вычисляется в виде комбинации случайно выбранного значения и заранее заданного параметра η, называемого индексом распределения, или **коэффициентом скученности** (crowding factor). Чем больше значение η, тем больше потомки похожи на своих родителей. Обычно η выбирается в диапазоне от 10 до 20.

- Нормально распределенная (или гауссова) мутация, когда исходное значение заменяется случайным числом, выбираемым из нормального распределения с заданными средним и стандартным отклонением.

Использование DEAP совместно с непрерывными функциями

При решении задач непрерывного поиска каркас DEAP можно использовать для оптимизации функций почти так же, как мы делали до сих пор. Нужно лишь внести несколько незначительных изменений.

Для кодирования хромосом используется список (или массив) чисел с плавающей точкой. Но следует помнить, что встроенные в DEAP генетические операторы не будут работать с индивидуумами, расширяющими класс `numpy.ndarray`, вследствие особенностей операций срезки и сравнения объектов.

Чтобы можно было работать с индивидуумами на основе `numpy.ndarray`, мы должны переопределить генетические операторы. Это описано в документации по DEAP, в разделе «Inheriting from NumPy». По данной причине, а также из соображений производительности обычно при работе с DEAP отдают предпочтение обыкновенным спискам или массивам вещественных чисел, встроенным в сам язык Python.

Что касается вещественных генетических операторов, то каркас DEAP предлагает несколько готовых реализаций в модулях `crossover` и `mutation`:

- `cxBlend()` – реализация скрещивания смешением, коэффициент α передается в параметре `alpha`;
- `cxSimulatedBinary()` – реализация имитации двоичного скрещивания, величина η передается в параметре `eta`;
- `mutGaussian()` – реализация нормально распределенной мутации, среднее и стандартное отклонение передаются в аргументах `mu` и `sigma`.

Кроме того, поскольку непрерывные функции обычно оптимизируются в ограниченной области, а не во всем пространстве, DEAP предоставляет два

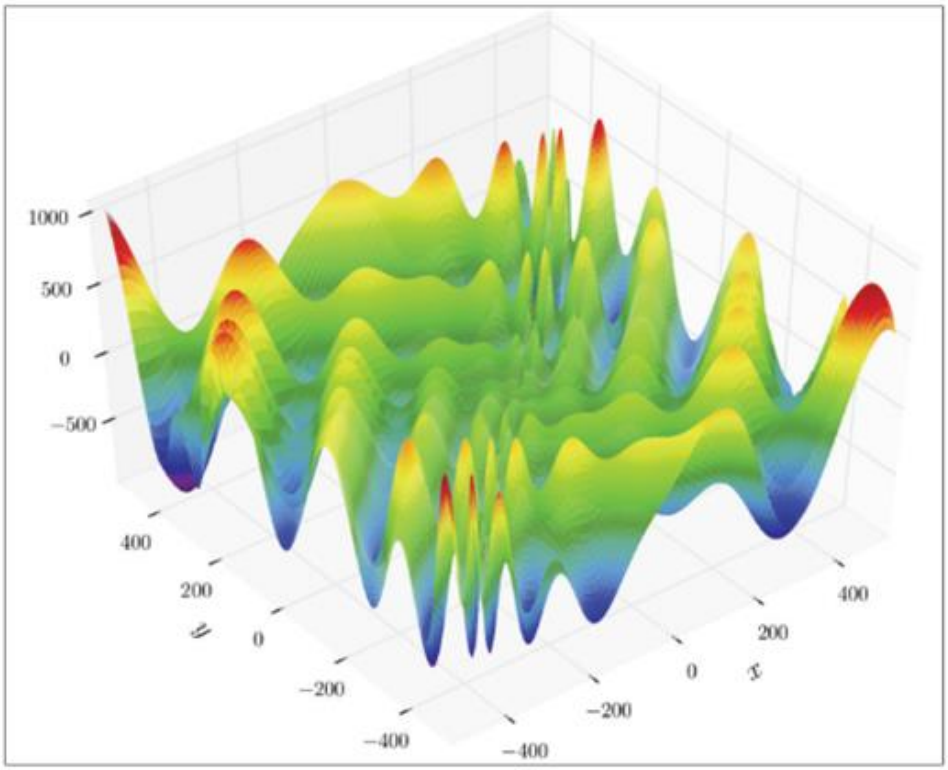
оператора, которые принимают параметры, описывающие границу, и гарантируют, что результирующие индивидуумы не выйдут за эти границы:

- `cxSimulatedBinaryBounded()` – ограниченный вариант оператора `cxSimulatedBinary()`, принимающий аргументы `low` и `up` – нижнюю и верхнюю границы области поиска соответственно;
- `mutPolynomialBounded()` – ограниченный оператор мутации, в котором распределение вероятностей задается полиномиальной (а не гауссовой) функцией. Этот оператор также принимает аргументы `low` и `up` – нижнюю и верхнюю границы области поиска. Кроме того, он принимает коэффициент скученности (параметр `eta`) – чем он больше, тем ближе мутант к исходному значению. Соответственно, при малых значениях `eta` мутант будет сильно отличаться от оригинала.

В следующем разделе мы продемонстрируем использование ограниченных операторов на примере оптимизации классической тестовой функции.

Оптимизация функции Eggholder

Функция Eggholder (она называется так потому, что ее форма напоминает подставку для яиц) часто используется для тестирования алгоритмов оптимизации. Нахождение единственного глобального минимума этой функции считается трудной задачей из-за большого количества локальных минимумов.



Функция Eggholder

Источник: https://en.wikipedia.org/wiki/File:Eggholder_function.pdf.
Автор: Gaortizg. Публикуется по лицензии Creative Commons CC BY-SA 3.0:
https://creativecommons.org/licenses/by-sa/3.0/deed.en

Математически функция описывается выражением

$$f(x,y) = -(y+47)\cdot\sin\sqrt{\left|\frac{x}{2}+(y+47)\right|} - x\cdot\sin\sqrt{|x-(y+47)|}$$

и обычно вычисляется в области поиска, ограниченной интервалом [–512, 512] по каждому измерению.

Известно, что глобального минимума эта функция достигает в точке

x=512, *y = 404.2319*,

в которой равна –959.6407.

В следующем подразделе мы попробуем найти глобальный минимум, применив генетический алгоритм.

Оптимизация функции Eggholder с помощью генетического алгоритма

Для оптимизации функции Eggholder с помощью генетического алгоритма мы написали программу, которая находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter06/01-optimize-eggholder.py.

Ниже описаны основные шаги программы.

1. Сначала определяются константы, а именно: количество измерений 2 (т. к. функция определена на плоскости *xy*) и границы области:

```
DIMENSIONS = 2                        # количество измерений
BOUND_LOW, BOUND_UP = -512.0, 512.0 # границы, одинаковые для всех измерений
```

2. Поскольку мы работаем с вещественными числами, ограниченными некоторой областью, далее определим вспомогательную функцию, порождающую числа с плавающей точкой, равномерно распределенные в этой области:



В этой функции предполагается, что верхняя и нижняя границы по всем измерениям одинаковы.

```
def randomFloat(low, up):
    return [random.uniform(l, u) for l, u in zip([low] *
DIMENSIONS, [up] * DIMENSIONS)]
```

3. Далее определим оператор `attrFloat`. В нем только что определенная функция используется для порождения одного случайного числа с плавающей точкой в заданном диапазоне. Этот оператор будет затем использован оператором `individualCreator` для создания случайных индивидуумов. А тот, в свою очередь, – оператором `populationCreator`, порождающим заданное количество индивидуумов.

```
toolbox.register("attrFloat", randomFloat, BOUND_LOW, BOUND_UP)
toolbox.register("individualCreator", tools.initIterate,
creator.Individual, toolbox.attrFloat)
toolbox.register("populationCreator", tools.initRepeat, list,
toolbox.individualCreator)
```

4. Мы собираемся минимизировать функцию Eggholder, поэтому ее и будем использовать как функцию приспособленности. Поскольку индивидуум представлен списком чисел с плавающей точкой длины 2, выделим из него значения *x* и *y* и вычислим функцию:

```
def eggholder(individual):
    x = individual[0]
    y = individual[1]
    f = (-(y + 47.0) * np.sin(np.sqrt(abs(x/2.0 + (y + 47.0))))
- x * np.sin(np.sqrt(abs(x - (y + 47.0)))))
    return f,   # вернуть кортеж

toolbox.register("evaluate", eggholder)
```

5. Далее разберемся с генетическими операторами. Поскольку оператор отбора не зависит от типа индивидуума, а у нас уже есть удачный опыт использования турнирного отбора размера 2 в сочетании с элитизмом, то и будем продолжать в том же духе. С другой стороны, операторы скрещивания и мутации нужно специализировать для чисел с плавающей точкой в заданных границах, поэтому воспользуемся предоставленным DEAP операторами `cxSimulatedBinaryBounded` для скрещивания и `mutPolynomialBounded` для мутации:

```
# Генетические операторы
toolbox.register("select", tools.selTournament, tournsize=2)
toolbox.register("mate", tools.cxSimulatedBinaryBounded,
low=BOUND_LOW, up=BOUND_UP, eta=CROWDING_FACTOR)
toolbox.register("mutate", tools.mutPolynomialBounded,
low=BOUND_LOW, up=BOUND_UP, eta=CROWDING_FACTOR,
indpb=1.0/DIMENSIONS)
```

6. Как и раньше, воспользуемся модифицированным вариантом простого алгоритма генетического поиска, встроенного в DEAP, в который мы добавили элитизм, – будем сохранять лучших индивидуумов (в зале славы) и копировать их в следующее поколение без изменения:

```
population, logbook = elitism.eaSimpleWithElitism(population,
toolbox, cxpb=P_CROSSOVER, mutpb=P_MUTATION,
ngen=MAX_GENERATIONS, stats=stats, halloffame=hof, verbose=True)
```

7. Начнем с задания параметров генетического алгоритма. Поскольку оптимизировать функцию Eggholder нелегко, зададим относительно большую популяцию, при небольшом числе измерений на это можно пойти:

```
# Параметры генетического алгоритма
POPULATION_SIZE = 300
P_CROSSOVER = 0.9
P_MUTATION = 0.1
MAX_GENERATIONS = 300
HALL_OF_FAME_SIZE = 30
```

8. В дополнение к предыдущим параметрам появляется один новый – коэффициент скученности (eta), задействованный в операциях скрещивания и мутации:

```
CROWDING_FACTOR = 20.0
```

Можно также определить отдельные коэффициенты скученности для скрещивания и для мутации.

Вот теперь мы готовы запустить программу. Ниже показаны результаты, полученные с заданными параметрами:

```
-- Лучший индивидуум = [512.0, 404.23180541839946]
-- Лучшая приспособленность = -959.6406627208509
```

Это означает, что мы нашли глобальный минимум.

Из показанных ниже графиков статистики видно, что алгоритм сразу же нашел какой-то локальный минимум, а затем производил небольшие улучшения, пока в конечном итоге не вышел на глобальный минимум.

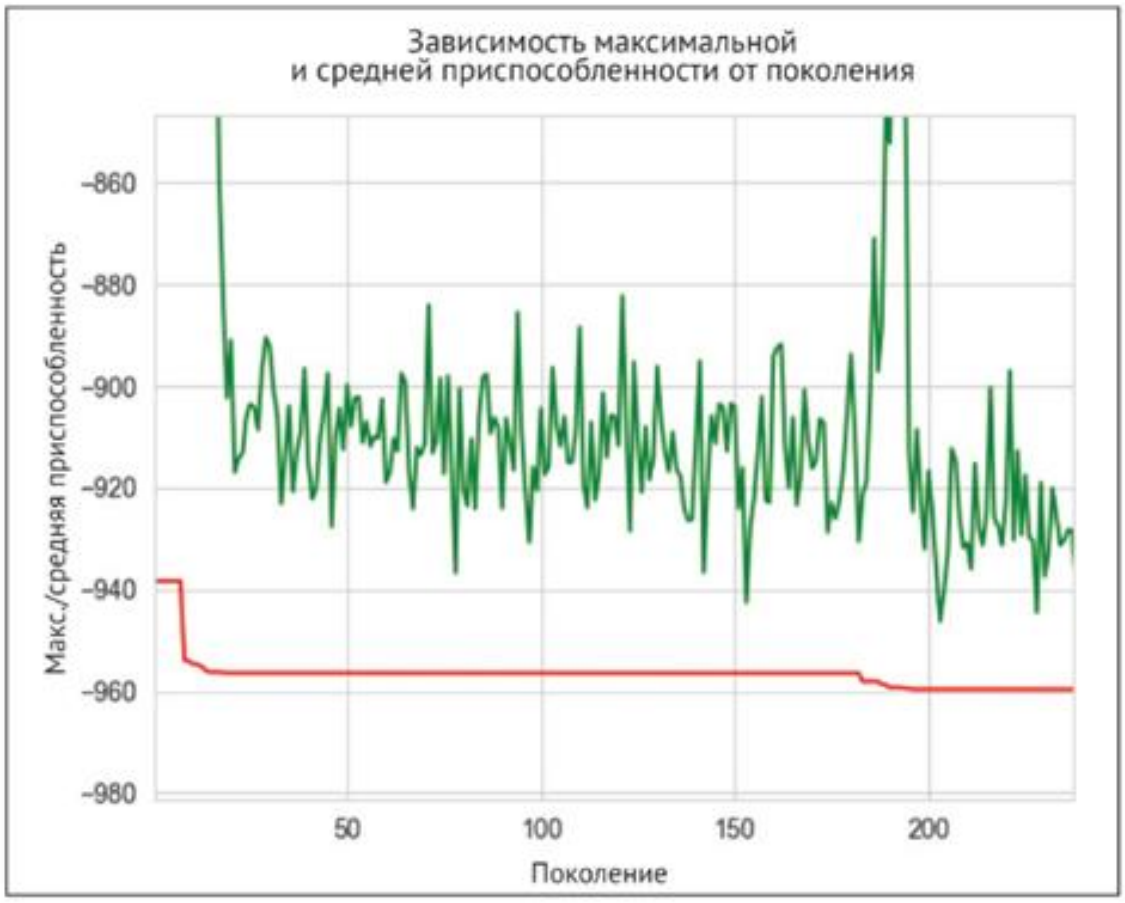


Статистика первой программы оптимизации функции Eggholder

Интересно, что произошло в окрестности поколения 180 – мы изучим этот феномен в следующем подразделе.

Повышение скорости сходимости посредством увеличения частоты мутаций

Увеличив масштаб в нижней части графика приспособленности, мы заметим довольно сильный скачок лучшего найденного результата (красная линия) в районе поколения 180, сопровождаемый резкими колебаниями средних результатов (зеленая линия).



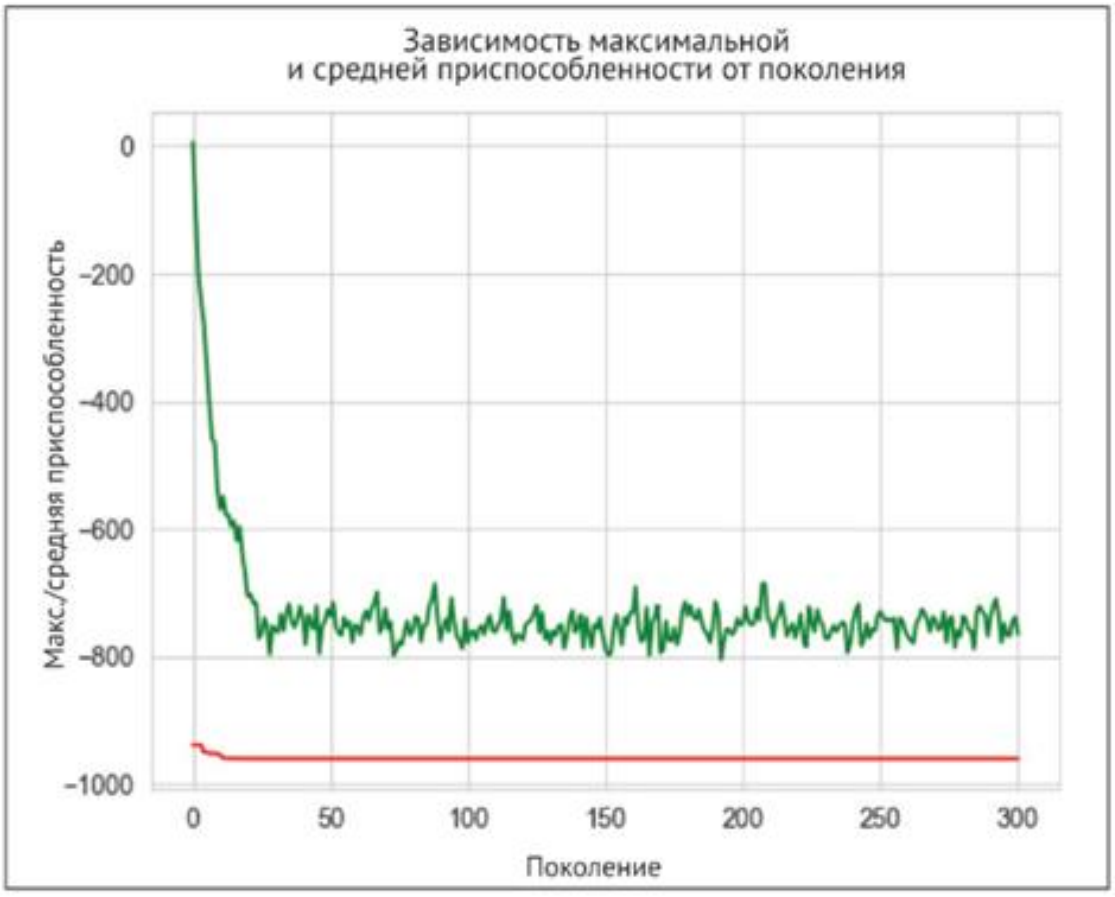
Увеличенная часть графиков статистики первой версии программы

Это наблюдение можно объяснить тем, что увеличение шума приводит к более быстрому получению хороших результатов. Возможно, это еще одно проявление уже знакомой нам дилеммы **исследования–использования** – увеличение степени исследования (которое выглядит на рисунке как шум) может ускорить отыскание глобального минимума. Простой способ повысить степень исследования – увеличить вероятность мутации. По счастью, использование элитизма – сохранение лучших результатов – не даст нам зайти в исследовании слишком далеко и сделать поиск случайным.

Для проверки этой идеи увеличим вероятность мутации с 0.1 до 0.5:

```
P_MUTATION = 0.5
```

Модифицированная таким образом программа находит глобальный минимум гораздо быстрее – это видно по графику, на котором красная линия (лучший результат) достигает оптимума быстро, а зеленая линия (средняя приспособленность) сильнее флуктуирует и дальше отстоит от лучшего результата.

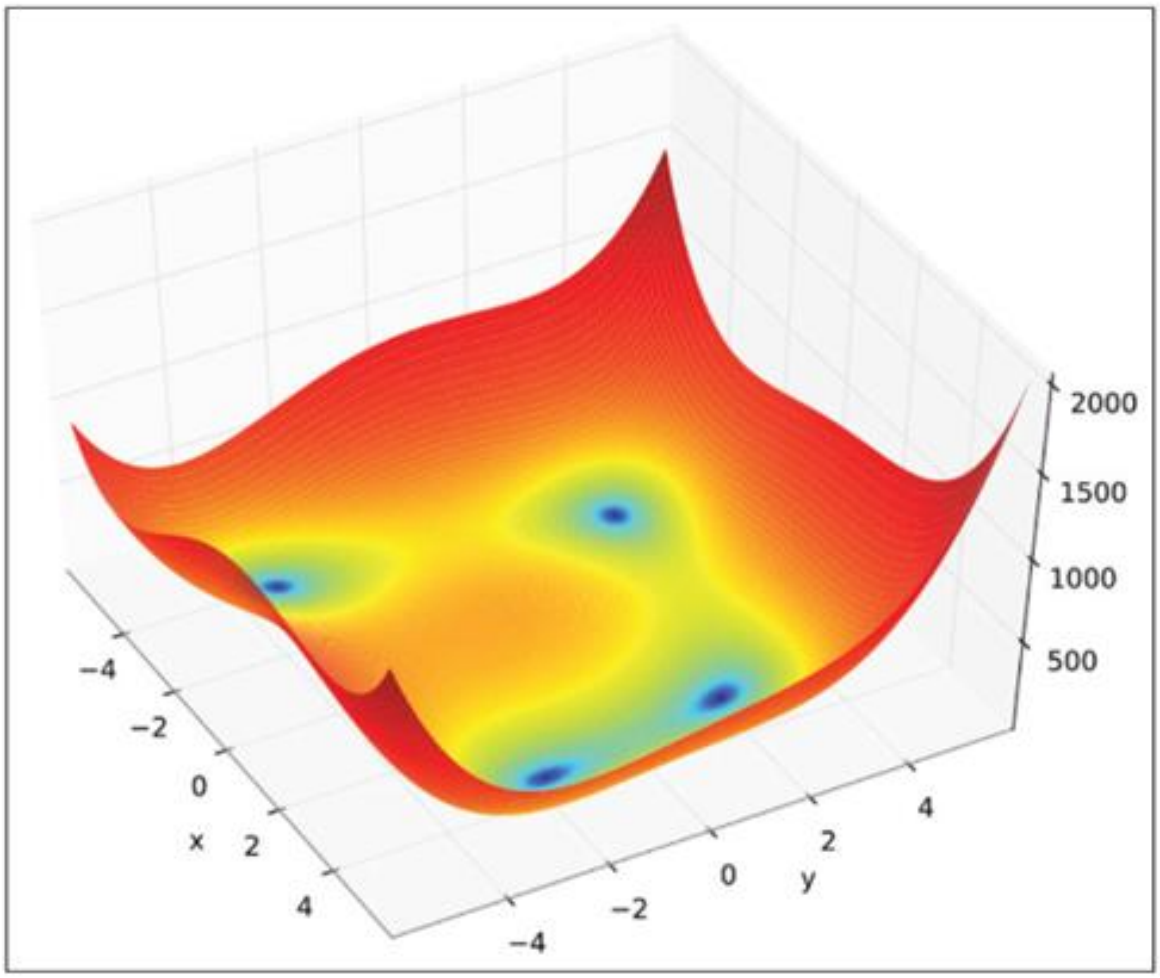


Статистика программы оптимизации функции Eggholder
с увеличенной вероятностью мутации

Будем иметь в виду эту идею в следующем разделе, работая с тестовой функцией Химмельблау.

Оптимизация функции Химмельблау

На рисунке ниже изображена функция Химмельблау, которая также часто используется для тестирования алгоритмов оптимизации.



Функция Химмельблау

Источник: https://commons.wikimedia.org/wiki/File:Himmelblau_function.svg.
Автор: Morn the Gorn. Является общественным достоянием

Математически эта функция описывается следующей формулой:

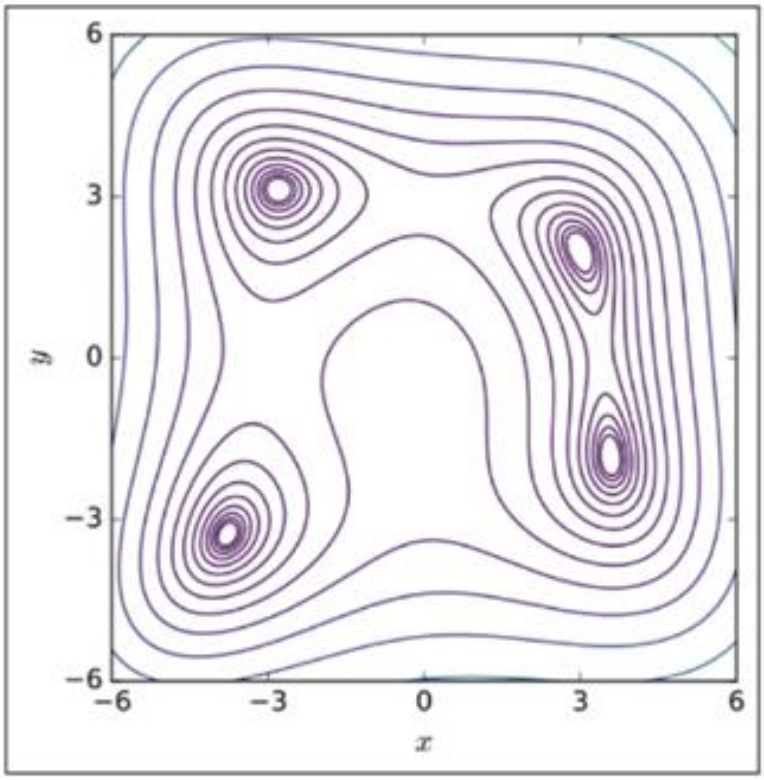
$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$.

Обычно она вычисляется в области, ограниченной интервалом [−5, 5] по каждому измерению.

Хотя эта функция кажется проще функции Eggholder, особый интерес представляет ее *многомодальность*, т. е. наличие нескольких глобальных минимумов. Точнее, у функции имеется четыре глобальных минимума, равных 0, в следующих точках:

- $x = 3.0, y = 2.0$;
- $x = -2.805118, y = 3.131312$;
- $x = -3.779310, y = -3.283186$;
- $x = 3.584458, y = -1.848126$.

Эти точки показаны на следующей диаграмме линий уровня:



Линии уровня функции Химмельблау

Источник: https://commons.wikimedia.org/wiki/File:Himmelblau_contour.svg.
Автор: Nicoguaro. Публикуется по лицензии Creative Commons CC BY 4.0:
https://creativecommons.org/licenses/by/4.0/deed.en

В процессе оптимизации многомодальных функций часто требуется найти все минимумы (или хотя бы большинство их). Но в следующем подразделе мы начнем с нахождения одного.

Оптимизация функции Химмельблау с помощью генетического алгоритма

Программа нахождения одного минимума функции Химмельблау находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter06/02-optimize-himmelblau.py.

Эта программа похожа на программу оптимизации функции Eggholder, но имеет несколько отличий:

1. Область оптимизации ограничена интервалом [−5.0, 5.0] по всем измерениям:

```
BOUND_LOW, BOUND_UP = -5.0, 5.0 # границы по всем измерениям
```

2. В качестве функции приспособленности используется функция Химмельблау:

```
def himmelblau(individual):
    x = individual[0]
    y = individual[1]
    f = (x ** 2 + y - 11) ** 2 + (x + y ** 2 - 7) ** 2
    return f,   # вернуть кортеж

toolbox.register("evaluate", himmelblau)
```

3. Поскольку оптимизируемая функция имеет несколько минимумов, интересно в конце прогона понаблюдать за распределением найденных решений. Поэтому добавим диаграмму рассеяния, показывающую все четыре глобальных минимума и окончательную популяцию на одной и той же плоскости *xy*:

```
plt.figure(1)
globalMinima = [[3.0, 2.0], [-2.805118, 3.131312], [-3.779310, -3.283186],
[3.584458, -1.848126]]
plt.scatter(*zip(*globalMinima), marker='X', color='red', zorder=1)
plt.scatter(*zip(*population), marker='.', color='blue', zorder=0)
```

4. Также распечатаем содержимое зала славы – лучших найденных индивидуумов:

```
print("- Лучшие решения:")
for i in range(HALL_OF_FAME_SIZE):
    print(i, ": ", hof.items[i].fitness.values[0], " -> ",
hof.items[i])
```

Результаты, напечатанные после прогона программы, показывают, что мы нашли один из четырех минимумов ($x = 3.0$, $y = 2.0$):

```
-- Лучший индивидуум = [2.9999999999987943, 2.0000000000007114]
-- Лучшая приспособленность = 4.523490304795033e-23
```

Из распечатки зала славы видно, что все его члены представляют одно и то же решение:

```
- Лучшие решения:
0 : 4.523490304795033e-23 -> [2.9999999999987943, 2.0000000000007114]
1 : 4.523732642865117e-23 -> [2.9999999999987943, 2.000000000000697]
2 : 4.523900512465748e-23 -> [2.9999999999987943, 2.0000000000006937]
3 : 4.5240633333565856e-23 -> [2.9999999999987943, 2.00000000000071]
...
```

Следующий рисунок, на котором изображено распределение популяции, подтверждает, что генетический алгоритм сошелся к одному из четырех минимумов функции – находящемуся в точке (x = 3.0, y = 2.0):

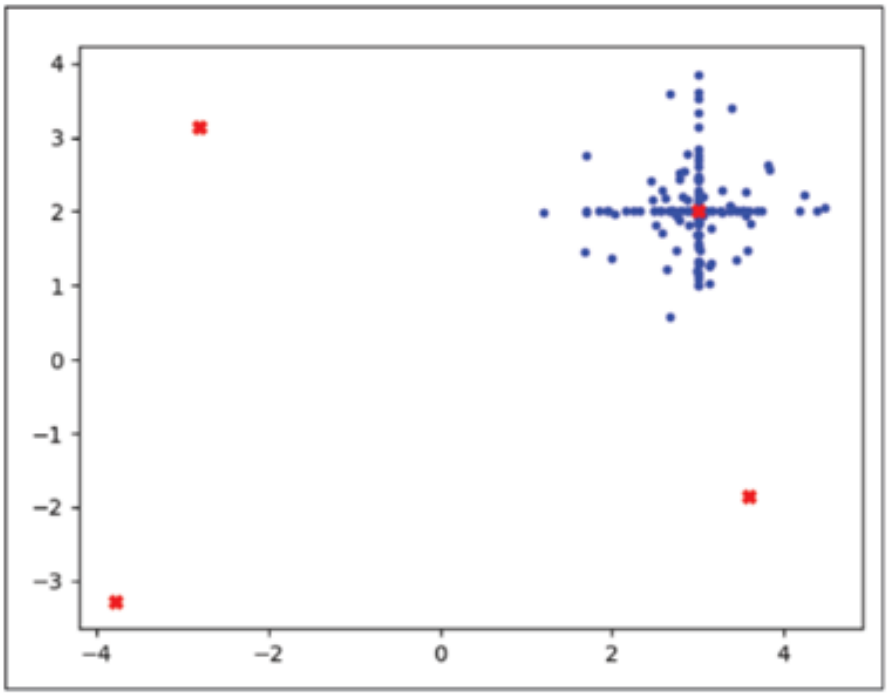


Диаграмма рассеяния популяции в конце первого прогона
и все четыре минимума функции

Кроме того, видно, что для многих индивидуумов в популяции одна из координат x и y совпадает с координатой найденного минимума.

Полученные результаты типичны для генетического алгоритма – он находит глобальный минимум и сходится к нему. Поскольку в данном случае минимумов несколько, ожидается сходимость к какому-то одному. К какому именно, зависит от случайной инициализации алгоритма. Как вы, наверное, помните, мы задавали фиксированное начальное значение генератора случайных чисел (42):

```
RANDOM_SEED = 42
random.seed(RANDOM_SEED)
```

Это сделано для того, чтобы обеспечить повторяемость результатов, но на практике обычно задаются разные начальные значения для разных прогонов – для этого нужно либо закомментировать приведенные выше строки, либо явно задавать различные значения константы.

Например, если задать начальное значение 13, то будет найдено решение (x = −2.805118, y = 3.131312), показанное на следующем рисунке.

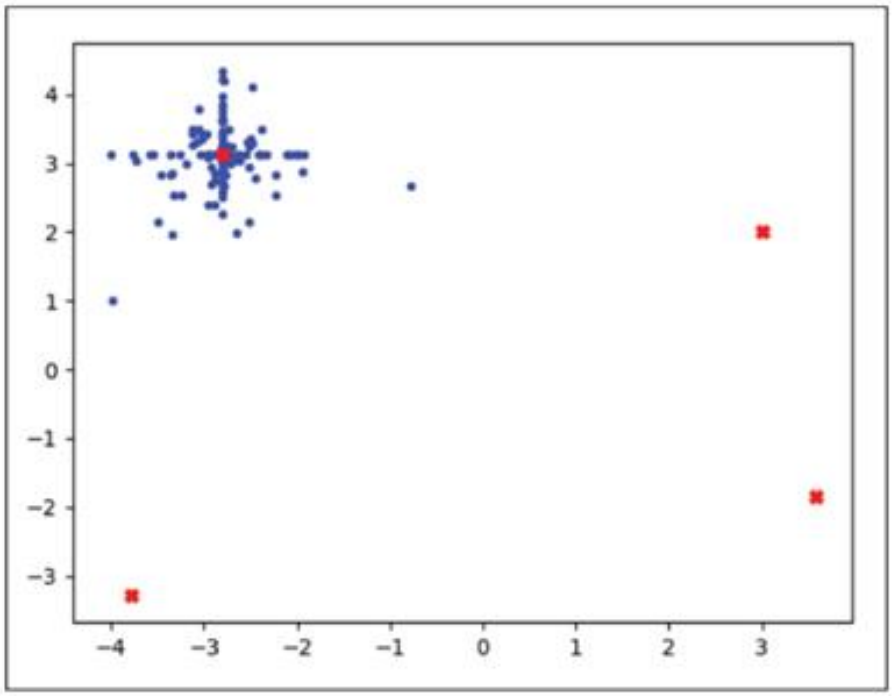


Диаграмма рассеяния популяции в конце второго прогона
и все четыре минимума функции

Если далее задать начальное значение 17, то программа найдет решение (x = 3.584458, y = −1.848126), как видно по следующему рисунку.

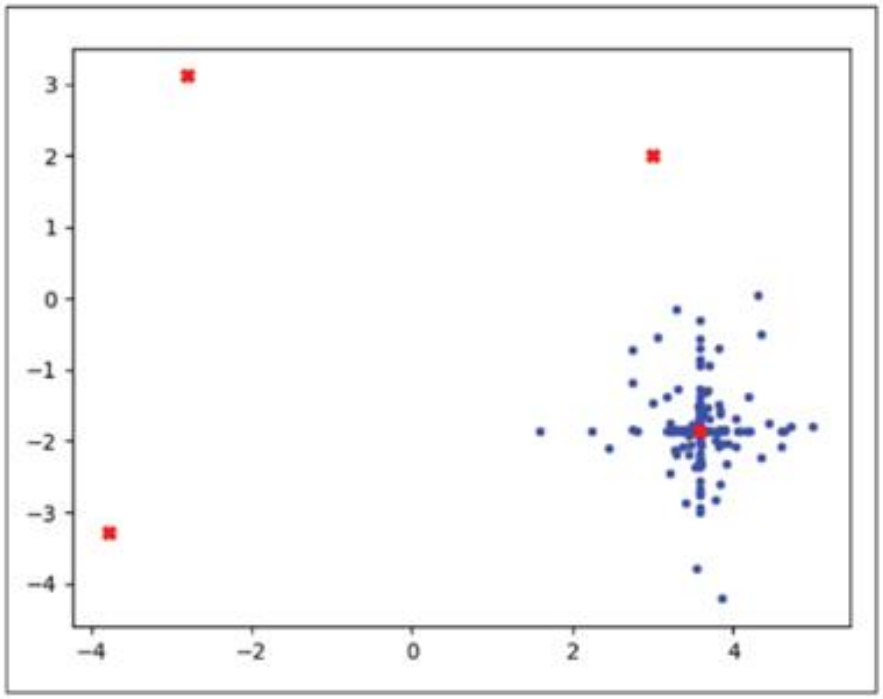


Диаграмма рассеяния популяции в конце третьего прогона
и все четыре минимума функции

Но что, если мы хотим найти все четыре глобальных минимума за один прогон? Как показано в следующем подразделе, генетические алгоритмы предлагают и такую возможность.

ОПТИМИЗАЦИЯ МЕТОДОМ РОЯ ЧАСТИЦ

Для работы нам понадобится Python 3 и следующие библиотеки:

- deap;
- numpy;
- networkx.

Идея оптимизации методом роя частиц (particle swarm optimization - PSO) подсмотрена у природных сообществ отдельных организмов - стай птиц, косяков рыб ит. п., обычно называемых роями. Организмы взаимодействуют в рое без какой-либо централизованной координации, но совместно достигают общей цели. Это наблюдаемое поведение положило начало вычислительному методу, который может решить задачу или выполнить оптимизацию, пользуясь группой потенциальных решений, представленных частицами, - по аналогии с организмами в рое. Частицы перемещаются в пространстве поиска, отыскивая лучшее решение, а их перемещение подчиняется простым правилам, формулируемым в терминах положения и скорости (векторной).

Алгоритм PSO итеративный, на каждой итерации вычисляется положение каждой частицы и обновляются ее лучшее на текущий момент положение, а также положение лучшей частицы во всей группе. Затем обновляются скорости всех частиц на основе следующей информации:

- текущая скорость и направление движения частицы - представляют инерцию;
- текущее лучшее положение (локально лучшее) - представляет когнитивную силу.

- положение лучшей частицы во всей группе (глобально лучшее) – представляет социальную силу.

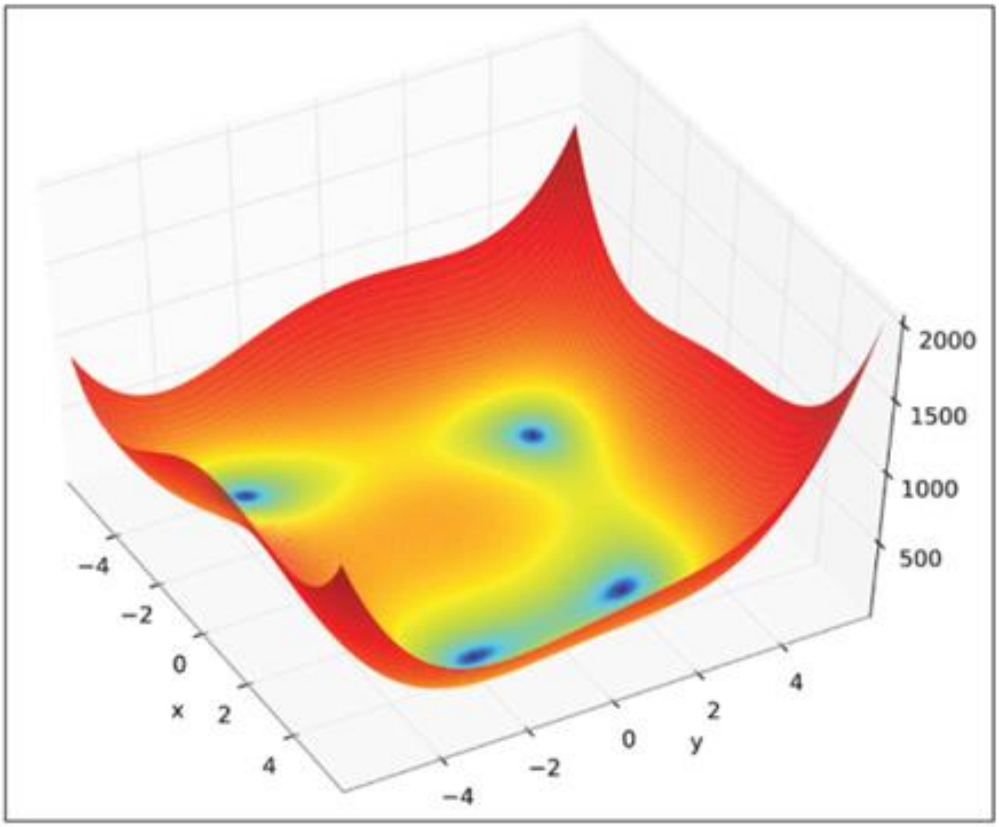
Затем производится обновление положения частиц, исходя из пересчитанной скорости.

Процесс продолжается до выполнения условия остановки, например достижения предельного числа итераций. Наилучшее положение группы на данный момент считается решением, найденным алгоритмом.

Этот простой, но эффективный процесс подробно иллюстрируется в следующем разделе, где мы напишем программу, которая будет оптимизировать функцию с помощью алгоритма PSO.

Пример применения PSO – оптимизация функции

Для демонстрации воспользуемся алгоритмом PSO, чтобы найти положение минимума (или минимумов) тестовой функции Химмельблау, которую мы уже оптимизировали в главе 6 с помощью генетического алгоритма. Эта функция изображена на рисунке ниже.



Функция Химмельблау

Источник: https://commons.wikimedia.org/wiki/File:Himmelblau_function.svg.
Автор: Morn the Gorn. Является общественным достоянием

Напомним, что математически функция описывается формулой

$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$.

У нее четыре глобальных минимума, равных 0. Они отмечены синим цветом и находятся в точках:

- $x = 3.0, y = 2.0$;
- $x = -2.805118, y = 3.131312$;
- $x = -3.779310, y = -3.283186$;
- $x = 3.584458, y = -1.848126$.

Мы будем искать только один из этих минимумов.

Реализация оптимизации методом роя частиц

Для нахождения минимума функции Химмельблау методом роя частиц мы написали на Python программу, которая находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter12/03-pso-himmelblau.py.

Ниже описаны основные части программы.

1. Для начала зададим значения различных констант. Размерность задачи в нашем случае равна 2, а она, в свою очередь, определяет размерность положения и скорости частиц. Следующая константа – размер популяции, т. е. количество частиц в рое, и, наконец, количество поколений, или итераций алгоритма.

```
DIMENSIONS = 2
POPULATION_SIZE = 20
MAX_GENERATIONS = 500
```

2. Далее зададим несколько дополнительных констант, влияющих на порядок создания и обновления частиц. Их роль станет ясна, когда мы будем рассматривать код программы:

```
MIN_START_POSITION, MAX_START_POSITION = -5, 5
MIN_SPEED, MAX_SPEED = -3, 3
MAX_LOCAL_UPDATE_FACTOR = MAX_GLOBAL_UPDATE_FACTOR = 2.0
```

3. Поскольку мы стремимся найти минимум функции Химмельблау, определим единственную цель – минимизирующую стратегию приспособления:

```
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
```

4. Теперь создадим класс `Particle`. Поскольку этот класс представляет положение в непрерывном пространстве, мы могли бы унаследовать его от обычного списка чисел с плавающей точкой. Однако здесь мы решили воспользоваться N-мерным массивом (`ndarray`) из библиотеки `numpy`, так как он особенно удобен для выполнения поэлементных алгебраических операций, в частности сложения и умножения, которые понадобятся при обновлении положения частицы. Помимо текущего положения, методу создания объекта класса `Particle` передается несколько дополнительных атрибутов:

- fitness – определенная выше минимизирующая стратегия приспособления;
- speed – здесь будут храниться компоненты текущего вектора скорости частицы. Начальное значение равно None, но позже скорость будет инициализирована с помощью другого массива ndarray;
- best – представляет лучшее найденное на данный момент положение этой частицы (локально лучшее).

В итоге определение метода создания объекта класса Particle выглядит следующим образом:

```
creator.create("Particle", np.ndarray,
fitness=creator.FitnessMin, speed=None, best=None)
```

5. Для конструирования одной частицы нам понадобится вспомогательная функция, которая создает частицу и инициализирует ее случайным образом. Мы воспользуемся функцией random.uniform() из библиотеки numpy, чтобы сгенерировать случайное положение и скорость частицы в заданных пределах:

```
def createParticle():
    particle = creator.Particle(np.random.uniform(
                                    MIN_START_POSITION,
                                    MAX_START_POSITION,
                                    DIMENSIONS))
    particle.speed = np.random.uniform(MIN_SPEED, MAX_SPEED,
                                    DIMENSIONS)
    return particle
```

6. Эта функция используется в определении оператора, который создает экземпляр частицы, а он, в свою очередь, применяется в операторе создания популяции:

```
toolbox.register("particleCreator", createParticle)
toolbox.register("populationCreator", tools.initRepeat, list,
toolbox.particleCreator)
```

7. Далее определим сердце алгоритма – метод updateParticle(), отвечающий за обновление положения и скорости каждой частицы в популяции. Ему передается одна частица и лучшее найденное на данный момент положение.
   Первым делом метод создает два случайных множителя – для локального и глобального обновлений – в заранее заданном диапазоне. Затем он вычисляет два обновления скорости (локальное и глобальное) и прибавляет их к текущей скорости частицы.
   Заметим, что все участвующие в вычислениях значения имеют тип массива ndarray, в нашем случае двумерного, а вычисления производятся поэлементно.
   Обновленная скорость частицы – это комбинация ее исходной скорости (представляющей инерцию), ее лучшего известного положения

(когнитивная сила) и лучшего известного положения частицы во всей популяции (социальная сила):

```
def updateParticle(particle, best):

    localUpdateFactor = np.random.uniform(0,
                                          MAX_LOCAL_UPDATE_FACTOR,
                                          particle.size)
    globalUpdateFactor = np.random.uniform(0,
                                          MAX_GLOBAL_UPDATE_FACTOR,
                                          particle.size)

    localSpeedUpdate = localUpdateFactor * (particle.best - particle)
    globalSpeedUpdate = globalUpdateFactor * (best - particle)

    particle.speed = particle.speed + (localSpeedUpdate + lobalSpeedUpdate)
```

8. Метод `updateParticle()` далее проверяет, что новая скорость не выходит за установленные пределы, и обновляет положение частицы с учетом пересчитанной скорости. Выше мы отмечали, что положение и скорость имеют тип `ndarray`, содержащий по одному элементу в каждом измерении:

```
particle.speed = np.clip(particle.speed, MIN_SPEED, MAX_SPEED)
particle[:] = particle + particle.speed
```

9. Теперь зарегистрируем метод `updateParticle()` как оператор инструментария, который будет впоследствии использоваться в главном цикле:

```
toolbox.register("update", updateParticle)
```

10. Нам еще нужно определить подлежащую оптимизации функцию Химмельблау и зарегистрировать ее как оператор вычисления приспособленности:

```
def himmelblau(particle):
    x = particle[0]
    y = particle[1]
    f = (x ** 2 + y - 11) ** 2 + (x + y ** 2 - 7) ** 2
    return f, # return a tuple

toolbox.register("evaluate", himmelblau)
```

11. Вот мы, наконец, и добрались до метода `main()`, где можем начать с создания популяции частиц:

```
population = toolbox.populationCreator(n=POPULATION_SIZE)
```

12. Прежде чем входить в главный цикл, нужно создать объект `stats`, необходимый для вычисления статистики, и объект `logbook`, в котором будет сохраняться статистика на каждой итерации:

```
stats = tools.Statistics(lambda ind: ind.fitness.values)
```

```
stats.register("min", np.min)
stats.register("avg", np.mean)

logbook = tools.Logbook()
logbook.header = ["gen", "evals"] + stats.fields
```

13. Главный цикл программы содержит внешний цикл по поколениям, в котором производится обновление частиц. На каждой итерации имеется два вспомогательных цикла, в каждом из которых обходятся все частицы в популяции. В первом цикле, показанном ниже, к каждой частице применяется оптимизируемая функция и при необходимости обновляются локально и глобально лучшие частицы:

```
particle.fitness.values = toolbox.evaluate(particle)

# локально лучшая
if particle.best is None or particle.best.size == 0 or
particle.best.fitness < particle.fitness:
    particle.best = creator.Particle(particle)
    particle.best.fitness.values = particle.fitness.values

# глобально лучшая
if best is None or best.size == 0 or best.fitness < particle.fitness:
   best = creator.Particle(particle)
   best.fitness.values = particle.fitness.values
```

14. Во втором внутреннем цикле вызывается оператор `update`. Ранее мы видели, что этот оператор обновляет скорость и положение частицы на основе комбинации инерции, когнитивной силы и социальной силы:

```
toolbox.update(particle, best)
```

15. В конце внешнего цикла мы сохраняем и печатаем статистику для текущего поколения:

```
logbook.record(gen=generation, evals=len(population),
**stats.compile(population))
print(logbook.stream)
```

16. По завершении внешнего цикла печатается информация о лучшем положении, найденном за все время работы алгоритма. Оно и считается решением задачи:

```
# печать информации о лучшем найденном решении
print("-- Лучшая частица = ", best)
print("-- Лучшая приспособленность = ", best.fitness.values[0])
```

Программа печатает следующий результат:

```
gen evals min      avg
0   20    8.74399  167.468
```

```
1   20   19.0871   357.577
2   20   32.4961   219.132
...
497 20    7.2162   412.189
498 20    6.87945  273.712
499 20   16.1034   272.385

-- Лучшая частица = [-3.77695478 -3.28649153]
-- Лучшая приспособленность = 0.0010248367255068806
```

Как видим, алгоритм смог найти один минимум рядом с точкой $x = -3.77$, $y = -3.28$. Статистика показывает, что лучший результат был достигнут в поколении 480. Также видно, что частицы колеблются, то приближаясь к лучшему результату, то отдаляясь от него.

Для нахождения других минимумов можно перезапустить алгоритм с иным начальным значением генератора случайных чисел.

Задание для самостоятельной работы

1. Оптимизировать функцию Eggholder с помощью генетического алгоритма. Перезапустить ГА с другими начальными значениями.
2. Выполнить изменение параметров функции Eggholder и повторить оптимизацию.
3. Оптимизировать функцию Химмельбау с помощью генетического алгоритма. Перезапустить ГА с другими начальными значениями.
4. Выполнить изменение параметров функции Химмельбау и повторить оптимизацию.
5. Выполнить оптимизацию функцию Химмельбау с использованием PSO. Перезапустить алгоритм с иным начальным значением генератора случайных чисел.